

A Repository-as-Source-of-Truth Architecture for AI-Assisted Course Material Authoring: An Experience Report from Five Undergraduate and Graduate Courses

Una arquitectura de repositorio como fuente única para la autoría de material docente asistida por IA: informe de experiencia en cinco asignaturas de pregrado y posgrado

Corresponding author

Julian Andrés Castaño Espinosa School of Engineering, Corporación Unificada Nacional de Educación Superior — CUN Bogotá, Colombia julian_castanoe@cun.edu.co ORCID: <https://orcid.org/0009-0003-6598-432X>

Authorship. This experience report is signed by a single author, who designed, implemented and operated the system described.

Institutional affiliation of the work: School of Engineering, CUN. Internal research call CUN 2026 — Thematic Research Groups, Phase II.

ABSTRACT

Generative AI is now routinely used to draft teaching material, but most reported practice consists of ad-hoc prompting: an instructor asks a chat assistant for a lesson plan, copies the output into a document, and the provenance of every factual claim is lost at that moment. This report describes a different arrangement, built and operated by one instructor across five courses over one academic term, in which the assistant never invents curricular facts because it is not permitted to hold them. Course codes, session topics, assessment weights, deadlines and institutional URLs live in a version-controlled repository as executable Python data; generation scripts read that data and emit the deliverables; and the assistant's role is confined to prose and structure. We describe the architecture, four safety patterns that emerged from operational failures rather than from design (dry-run-first ordering, machine-readable receipts, verification by export rather than by screen, and single-writer discipline), and the categories of error that the arrangement does and does not prevent. We report what the system covers — 5 courses and 50 planned sessions — and we are explicit that this is a single-instructor, single-institution experience report without a control condition, without learning-outcome measurement and without inter-rater assessment of material quality. The contribution is architectural and cautionary, not evaluative: we argue that the useful question is not whether AI can draft teaching material, which it plainly can, but where the authoritative copy of curricular fact should live so that drafting cannot silently corrupt it.

Keywords: AI-assisted authoring, educational technology, single source of truth, reproducibility, instructional design, higher education, large language models, workflow architecture

RESUMEN

La IA generativa se usa ya de forma corriente para redactar material docente, pero la práctica reportada suele ser improvisada: el docente pide un plan de clase a un asistente, copia la salida a un documento y en ese momento se pierde la trazabilidad de cada dato. Este informe describe una disposición distinta, construida y operada por un solo docente en cinco asignaturas durante un periodo académico, en la que el asistente no inventa hechos curriculares porque no se le permite custodiarlos. Los códigos de asignatura, los temas por sesión, los pesos de evaluación, las fechas y las URL institucionales viven en un repositorio versionado como datos ejecutables de Python; unos guiones de generación leen esos

datos y emiten los entregables; y el papel del asistente se limita a la prosa y la estructura. Describimos la arquitectura, cuatro patrones de seguridad que surgieron de fallos de operación y no del diseño, y las clases de error que la disposición evita y las que no. Reportamos la cobertura del sistema —5 asignaturas y 50 sesiones planificadas— y declaramos explícitamente que se trata de un informe de experiencia de un solo docente en una sola institución, sin condición de control, sin medición de resultados de aprendizaje y sin evaluación con jueces independientes de la calidad del material. La contribución es arquitectónica y cautelar, no evaluativa.

Palabras clave: autoría asistida por IA, tecnología educativa, fuente única de verdad, reproducibilidad, diseño instruccional, educación superior, modelos de lenguaje, arquitectura de flujo de trabajo

1. INTRODUCTION

The question of whether a large language model can draft a serviceable lesson plan has been settled in the affirmative by ordinary practice. Instructors do it daily. The question that remains open, and that this report addresses, is narrower and more consequential: **when an assistant drafts teaching material, where does the authoritative version of the underlying curricular fact live?**

The distinction matters because the two most damaging failure modes in AI-assisted course authoring are not stylistic. They are (i) the assistant confidently asserting an institutional fact that is wrong — an assessment weight, a submission deadline, a course code, a platform URL — and (ii) the same fact appearing in several generated documents with different values, so that a student reading the syllabus, the slide deck and the assignment brief receives three mutually inconsistent instructions. Neither failure is detectable by reading the generated document in isolation, because each document is internally coherent. Both are detectable immediately if the fact has exactly one authoritative location.

This report describes a system built on that single premise, operated by one instructor across five courses in one term, and it reports the operational consequences honestly, including the ones that were unwelcome.

1.1 Scope and what this report is not

This is an **experience report**, in the sense established in the software engineering literature: a structured account of a system built and operated in a real setting, offered for its architectural lessons rather than as evidence of effect. It is explicitly **not** a controlled study. There is no comparison group, no measurement of student learning outcomes, no blinded assessment of material quality, and no instrument measuring instructor time saved. Any of those would be a different and more expensive study, and several of them are proposed as future work in Section 6.

Readers looking for a claim that this approach improves learning will not find one here, because the data to support such a claim was not collected.

2. BACKGROUND AND RELATED WORK

2.1 Generative AI in instructional design

The rapid adoption of generative assistants in higher education teaching practice has been documented across disciplines, with most reported workflows following what may be called the *prompt-and-paste* pattern: a natural-language request produces a draft, the instructor edits it, and the result is saved as a document. The pattern's appeal is that it requires no infrastructure. Its cost is that the generated document becomes the only record of its own content: there is no separate, checkable statement of what the course's assessment weights actually are, against which the document could be validated.

2.2 Single source of truth as a software discipline

The principle that each datum should have exactly one authoritative representation is old in data management and in configuration management, where it is enforced to prevent update anomalies. Its application to instructional material appears less often, perhaps because course documents are usually authored by humans who are assumed to remember what they wrote. That assumption weakens sharply when a non-remembering assistant is the one drafting, and weakens further when the same facts must appear in a slide deck, a script, a learning-management-system configuration and an email.

2.3 Reproducible authoring pipelines

The practice of generating documents from data and templates rather than editing them by hand is well established in reproducible research, where the motivation is that a figure should not be able to disagree with the dataset that produced it. The architecture described here applies the same motivation to a different artefact class: a slide deck should not be able to disagree with the syllabus that produced it.

3. SYSTEM ARCHITECTURE

3.1 The layer separation

The system separates three layers that the prompt-and-paste pattern conflates:

1. **Curricular fact.** Course codes, session topics, credit hours, assessment components and their weights, submission deadlines, group codes, institutional URLs and the instructor's contact data. This layer is Python data in a version-controlled repository. It is the only place these values exist. Nothing generated may contradict it, because everything generated reads from it.
2. **Generation.** Scripts that read layer 1 and emit deliverables: session scripts for the instructor, slide decks, welcome emails, calendar-event automation, assessment configuration.
3. **Prose and structure.** The layer where a generative assistant is used: phrasing an explanation, organising a session's arc, drafting feedback. The assistant operates *within* this layer and is given layer 1 as read-only context.

The consequence that matters: a deadline cannot be wrong in the slide deck and right in the email, because neither document contains a deadline. Both contain a reference that is resolved at generation time from a single value.

3.2 Coverage

At the time of writing the system covers **five courses** — one graduate (Project I, Specialisation in Artificial Intelligence) and four undergraduate (Research in Science and Technology; Creativity and Innovative Thinking; Degree Project 2; Degree Project 3) — comprising **50 planned sessions**. Each course's official syllabus, where the institution has provided one, is parsed directly from the institutional document rather than transcribed, so that the topic list in the generated material is traceable to the source.

3.3 Syllabus parsing as a design constraint

A design decision worth reporting is that the syllabus reader **refuses to invent**. When an institution had not delivered the official syllabus for one course, the module returned an explicit "no syllabus available" state with a note, rather than a plausible topic list inferred by analogy with a sibling course. Generated material for that course carried the gap visibly. When the missing syllabus was later delivered, it turned out to use a document structure different from both formats the parser knew — an undergraduate institutional shell containing a graduate-style unit table — and the parser had to be extended rather than the document reinterpreted. Had the system filled the gap by inference, the discrepancy would never have surfaced.

4. SAFETY PATTERNS THAT EMERGED FROM FAILURE

The four patterns below were not designed in advance. Each was added after an operational failure, and each is reported here with the failure that motivated it, because the failure is the evidence.

4.1 *Dry-run-first ordering*

Any operation that writes to a system outside the repository — a learning management system, a student's shared document, a calendar — is implemented as an ordered sequence in which the non-writing modes come first and the writing mode requires an explicit confirmation flag. The motivating failure was a batch operation that was correct in intent and wrong in target.

4.2 *Machine-readable receipts*

When the system writes to an external service, it records what it wrote in a structured file. The motivating failure was the inability to distinguish the system's own output from a human collaborator's when both appeared in the same external document; without a receipt there is no principled way to undo only one's own changes.

4.3 *Verification by export, not by screen*

Operations whose result is rendered by a third-party application are verified by exporting the resulting document and inspecting its serialised form, rather than by reading the application's own display. The motivating failure was an interface that reported a save as successful while the underlying record remained empty — a discrepancy invisible on screen and unmistakable in the exported data.

4.4 *Single-writer discipline*

When several generation processes run concurrently, exactly one is permitted to write to any given output file; the others return data. The motivating failure was two processes editing the same document and silently overwriting each other's changes.

5. DISCUSSION

5.1 *What the architecture prevents, and what it does not*

The arrangement reliably prevents one class of error: **divergence of a curricular fact across generated documents**. This follows from construction rather than from vigilance, which is the point.

It does **not** prevent several other classes, and it is important to state them:

- **Wrong facts entered correctly.** If an assessment weight is entered incorrectly in layer 1, it will be propagated consistently and wrongly to every artefact. The architecture guarantees agreement, not truth.
- **Prose that is fluent and mistaken.** The assistant's contribution to layer 3 is not validated by the architecture. A conceptually wrong explanation, expressed clearly, passes through.
- **Institutional drift.** When the institution changes a deadline without notifying the instructor, the repository becomes confidently out of date.

5.2 *The cost side*

The arrangement is not free. It requires the instructor to be able to read and modify Python, to maintain a repository, and to accept that a change to a shared value requires regenerating the artefacts that depend on it. For an instructor teaching one course, the overhead very likely exceeds the benefit. The break-even

point is not measured here; identifying it is future work.

5.3 Threats to the validity of this report

The most serious threat is that the author is simultaneously the designer, the operator and the reporter of the system, with an evident interest in its appearing worthwhile. No independent party audited the material, and no student was surveyed. A second threat is that a single term in a single institution may not generalise: the institution's document formats, its learning management system and its administrative calendar are all specific.

6. CONCLUSIONS AND FUTURE WORK

We have described an architecture in which a generative assistant contributes prose and structure to course material while being structurally prevented from being the custodian of curricular fact, and we have reported four safety patterns that emerged from operational failure rather than from design. The system covers five courses and fifty planned sessions in one institution over one term.

We deliberately make no claim about learning outcomes, material quality or time saved, because the corresponding measurements were not taken.

Three studies would test what this report can only assert:

1. **A blinded quality comparison** in which instructors unaware of provenance rate material generated under this architecture against material produced by prompt-and-paste.
2. **A consistency audit** measuring, across a corpus of course documents from instructors not using the system, how frequently the divergence failure this architecture prevents actually occurs.
3. **A cost study** establishing the number of courses at which the maintenance overhead is repaid.

Until those are done, the honest summary is architectural: if an assistant is going to draft your teaching material, decide first where the authoritative copy of each curricular fact will live, and do not let that place be the draft.

ACKNOWLEDGEMENTS AND DECLARATIONS

Funding. This work was developed within the internal research call CUN 2026 — Thematic Research Groups, Phase II, of the Corporación Unificada Nacional de Educación Superior. No external funding.

Author contributions. Conceptualisation, methodology, software, writing of the original draft, and writing with review and editing: the single signing author.

Conflict of interest. The author declares no conflict of interest.

Data Availability Statement

Data Availability Statement / Declaración de disponibilidad de datos / Declaração de disponibilidade de dados. The system described is an institutional teaching repository that contains student-related administrative data — class rosters, group codes and institutional e-mail addresses — and therefore cannot be published as-is. No research dataset was generated by this work. A description of the architecture sufficient to reimplement it is given in Section 3, and the author will respond to reasonable requests for further methodological detail.

Declaration on the use of Artificial Intelligence

Declaration on the use of Artificial Intelligence / Declaración de uso de Inteligencia Artificial. Generative artificial intelligence assistants — Claude (Anthropic) and ChatGPT (OpenAI) — were used in

the preparation of this manuscript for drafting and editing prose, and for checking internal consistency between sections. They were also, and this is the subject of the report itself, used as the layer-3 component of the system described. No tool generated data, measurements or bibliographic references that were not individually verified by the author. The author accepts full responsibility for the content of this manuscript.

Personal data. This manuscript contains no personally identifying information about students. No student record, identifier or work product is reproduced.

Ethics. This report describes an authoring system operated by the author and does not involve research with human subjects. No data were collected from students, and no intervention was applied to them.

REFERENCES

Note on digital identifiers: entries are presented as verified with respect to authorship, year, title and source. DOIs and URLs will be incorporated after individual verification in Crossref. No DOI is transcribed that has not been verified.

Beck, K. (2003). *Test-Driven Development: By Example*. Addison-Wesley.

Brown, J. S., Collins, A., & Duguid, P. (1989). Situated cognition and the culture of learning. *Educational Researcher*, 18(1), 32–42.

Buckingham Shum, S., & Ferguson, R. (2012). Social learning analytics. *Educational Technology & Society*, 15(3), 3–26.

Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.

Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

Kitchenham, B., Pfleeger, S. L., Pickard, L. M., Jones, P. W., Hoaglin, D. C., El Emam, K., & Rosenberg, J. (2002). Preliminary guidelines for empirical research in software engineering. *IEEE Transactions on Software Engineering*, 28(8), 721–734.

Knuth, D. E. (1984). Literate programming. *The Computer Journal*, 27(2), 97–111.

Laurillard, D. (2012). *Teaching as a Design Science: Building Pedagogical Patterns for Learning and Technology*. Routledge.

Merrill, M. D. (2002). First principles of instruction. *Educational Technology Research and Development*, 50(3), 43–59.

Mishra, P., & Koehler, M. J. (2006). Technological pedagogical content knowledge: A framework for teacher knowledge. *Teachers College Record*, 108(6), 1017–1054.

Peng, R. D. (2011). Reproducible research in computational science. *Science*, 334(6060), 1226–1227.

Perkel, J. M. (2018). A toolkit for data transparency takes shape. *Nature*, 560(7719), 513–515.

Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164.

Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10), e1003285.

Selwyn, N. (2019). Should robots replace teachers? AI and the future of education. *Learning, Media and Technology*, 44(4), 1–14.

- Shute, V. J. (2008). Focus on formative feedback. *Review of Educational Research*, 78(1), 153–189.
- Wiggins, G., & McTighe, J. (2005). *Understanding by Design* (2nd ed.). Association for Supervision and Curriculum Development.
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510.
- Zawacki-Richter, O., Marín, V. I., Bond, M., & Gouverneur, F. (2019). Systematic review of research on artificial intelligence applications in higher education. *International Journal of Educational Technology in Higher Education*, 16(1), 39.